Programmation orientée objet (UAA14)

Atelier : les exceptions

Objectifs de l'atelier :

- ✔ Comprendre pourquoi une méthode ne devrait pas réagir seule à ses erreurs.
- ✔ Lancer une exception et lire le message d'erreur de PHP.
- ✔ Attraper une exception et suivre le chemin de l'exécution.
- ✔ Observer comment une exception remonte d'une méthode à l'autre.
- ✔ Créer ses propres types d'exception et les distinguer.
- ✔ Comprendre ce qui arrive quand un constructeur lance une exception.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie sur les exceptions. Tu n'as rien à apprendre par cœur : tu vas exécuter de petits programmes, les modifier, et découvrir par toi-même comment PHP réagit. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA14/fichiers/atelier_exceptions.zip

Comment travailler :

- ➢ télécharge les programmes de l'atelier (adresse ci-dessus) et décompresse l'archive dans le dossier `htdocs` de XAMPP : tu obtiens un dossier `atelier` qui contient un fichier par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ➢ ouvre chaque programme dans ton navigateur, par exemple http://localhost/atelier/section0.php ;
- ➢ **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- ➢ ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- ➢ à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : le problème

Voici une calculatrice qui se protège contre la division par zéro.

Fichier : section0.php

```php
<?php
class Calculatrice {
    function diviser($a, $b) {
        if ($b == 0) {
            echo "Erreur : division par zéro !<br>";
            return -1;
        }
        return $a / $b;
    }
}

$calc = new Calculatrice();
$moyenne = $calc->diviser(30, 3);
echo "Moyenne : $moyenne<br>";
$moyenne = $calc->diviser(30, 0);
echo "Moyenne : $moyenne<br>";
$resultat = $calc->diviser(-5, 5);
echo "Résultat : $resultat<br>";
```

Prédis

Qu'affiche ce programme ?

Exécute

Exécute le programme et compare avec ta prédiction.

Explique

La deuxième « moyenne » affichée vaut -1. Est-ce une moyenne ? Que risque le programme s'il continue ses calculs avec cette valeur ?

Le troisième appel retourne lui aussi -1. Comment le programme peut-il savoir si ce -1 est un vrai résultat ou le signal d'une erreur ?

Le message « Erreur : division par zéro ! » est affiché par la méthode `diviser`. Imagine que cette calculatrice serve dans trois programmes : un qui affiche ses messages en anglais, un qui ne doit rien afficher du tout, un qui doit proposer de recommencer. Qui devrait décider de la réaction à l'erreur : la méthode `diviser`, ou le programme qui l'appelle ?

✎ Ma règle : quels sont les deux défauts de cette façon de gérer les erreurs ?

Les exceptions ont été inventées pour résoudre ces deux problèmes. À toi de découvrir comment.

Section 1 : lancer une exception

Voici la même calculatrice, modifiée. Repère ce qui a changé dans la méthode `diviser`.

Fichier : section1.php

```php
<?php
class Calculatrice {
    function diviser($a, $b) {
        if ($b == 0) {
            throw new Exception("Division de $a par zéro");
        }
        return $a / $b;
    }
}

$calc = new Calculatrice();
echo "Début<br>";
echo "Résultat : " . $calc->diviser(30, 3) . "<br>";
echo "Résultat : " . $calc->diviser(30, 0) . "<br>";
echo "Fin<br>";
```

Prédis

Qu'affiche ce programme ? Le mot « Fin » sera-t-il affiché ?

Exécute

PHP affiche un message d'erreur qui ressemble à ceci (le chemin du fichier dépend de ton ordinateur) :

```
Fatal error: Uncaught Exception: Division de 30 par zéro in ...\
section1.php:5
Stack trace:
#0 ...\section1.php(14): Calculatrice->diviser(30, 0)
#1 {main}
  thrown in ...\section1.php on line 5
```

Explique

Dans ce message, retrouve : le type de l'erreur ; le message que tu as toi-même écrit ; la ligne où l'erreur a été lancée ; la ligne de ton script qui a appelé la méthode.

« Uncaught » signifie « non attrapée ». Qu'est-il arrivé au reste du programme ?

Lors du deuxième appel, le `return` de la méthode `diviser` a-t-il été exécuté ? Comment le sais-tu ?

Modifie

Change le texte du message de l'exception, puis remplace le 0 du deuxième appel par 6. Exécute après chaque changement. Qu'observes-tu ?

✎ Ma règle : que fait l'instruction throw new Exception("...") ?

Section 2 : attraper une exception

Ce programme reprend la classe `Calculatrice` de la section 1, avec son `throw`. Seul le script change :

Fichier : section2.php

```
$calc = new Calculatrice();
echo "Début<br>";
try {
    echo "A<br>";
    $r = $calc->diviser(30, 0);
    echo "B : $r<br>";
} catch (Exception $e) {
    echo "C : " . $e->getMessage() . "<br>";
}
echo "Fin<br>";
```

Prédis

Qu'est-ce qui sera affiché, et dans quel ordre ?

Exécute, puis explique

Pourquoi B n'est-il pas affiché ?

Cette fois, le programme s'est-il arrêté ? Pourquoi ?

Que contient la variable `$e` ? D'où vient le texte affiché après « C : » ?

Modifie

Remplace `diviser(30, 0)` par `diviser(30, 3)`. Prédis, puis exécute : qu'est-ce qui est affiché ?

Voici deux façons de diviser 100 par chacun des nombres 5, 0 et 2. Prédis l'affichage de chacune, puis vérifie.

Version 1 :

`Fichier : section2_boucle1.php (même classe Calculatrice)`

```
$calc = new Calculatrice();
foreach (array(5, 0, 2) as $diviseur) {
    try {
        echo "100 / $diviseur = " . $calc->diviser(100, $diviseur) .
"<br>";
    } catch (Exception $e) {
        echo "Impossible : " . $e->getMessage() . "<br>";
    }
}
echo "Fin<br>";
```

Version 2 :

Fichier : section2_boucle2.php (même classe Calculatrice)

```
$calc = new Calculatrice();
try {
    foreach (array(5, 0, 2) as $diviseur) {
        echo "100 / $diviseur = " . $calc->diviser(100, $diviseur) .
"<br>";
    }
} catch (Exception $e) {
    echo "Impossible : " . $e->getMessage() . "<br>";
}
echo "Fin<br>";
```

Quelle différence entre les deux versions explique la différence d'affichage ?

✎ Ma règle : que se passe-t-il quand une exception est lancée dans un bloc try ? Et quand aucune ne l'est ?

Section 3 : une exception qui remonte

Cette fois, l'exception est lancée tout au fond d'une chaine d'appels : le script appelle `afficherMoyenne`, qui appelle `moyenne`, qui appelle `diviser`.

Fichier : section3.php

```php
<?php
class Calculatrice {
    function diviser($a, $b) {
        if ($b == 0) {
            throw new Exception("Division de $a par zéro");
        }
        return $a / $b;
    }
    function moyenne($total, $nombre) {
        return $this->diviser($total, $nombre);
    }
}

class Bulletin {
    private $calc;
    function __construct() {
        $this->calc = new Calculatrice();
    }
    function afficherMoyenne($nom, $total, $nombre) {
        echo "$nom : " . $this->calc->moyenne($total, $nombre) . "<br>";
    }
}

$bulletin = new Bulletin();
try {
    $bulletin->afficherMoyenne("Zoé", 45, 3);
    $bulletin->afficherMoyenne("Malik", 0, 0);
    $bulletin->afficherMoyenne("Sam", 28, 2);
} catch (Exception $e) {
    echo "Problème : " . $e->getMessage() . "<br>";
}
echo "Fin<br>";
```

Prédis

Qu'affiche ce programme ? Fais bien attention à la ligne de Malik.

Exécute, puis explique

Par quelles méthodes l'exception est-elle passée avant d'être attrapée ?

Ni `moyenne` ni `afficherMoyenne` ne contiennent de `try`. Qu'ont-elles dû écrire pour laisser passer l'exception ?

Pourquoi « Malik : » n'est-il pas affiché, alors que l'`echo` commence par ce texte ?

Pourquoi Sam n'apparaît-il pas ?

Modifie

Supprime le `try` / `catch` du script, en gardant les trois appels. Place-le plutôt dans `afficherMoyenne`, autour de l'`echo`. Dans le `catch`, affiche « <nom> : pas de moyenne (<message>) ». Prédis, puis exécute.

Pour un bulletin, laquelle des deux versions est la meilleure ? Dans chaque version, qui décide de la réaction à l'erreur ?

Supprime maintenant tous les `try` / `catch`, et exécute. Dans la pile d'appels (Stack trace), à quoi correspondent les lignes #0, #1 et #2 ?

✎ Ma règle : que devient une exception qui n'est pas attrapée dans la méthode qui la lance ?

Section 4 : tes propres exceptions

Jusqu'ici, toutes les erreurs étaient de type `Exception`. Ce programme définit deux nouveaux types d'erreur.

Fichier : section4.php

```
<?php
class DivisionParZeroException extends Exception {}
class NombreNegatifException extends Exception {}

class Calculatrice {
    function diviser($a, $b) {
        if ($b == 0) {
            throw new DivisionParZeroException("Division de $a par zéro");
        }
        return $a / $b;
    }
    function racine($x) {
        if ($x < 0) {
            throw new NombreNegatifException("Pas de racine carrée pour
$x");
        }
        return sqrt($x);
    }
}

$calc = new Calculatrice();
foreach (array(16, -4, 0) as $x) {
    try {
        echo "1 / racine($x) = " . $calc->diviser(1, $calc->racine($x)) .
"<br>";
    } catch (DivisionParZeroException $e) {
        echo "Division impossible : " . $e->getMessage() . "<br>";
    } catch (NombreNegatifException $e) {
        echo "Racine impossible : " . $e->getMessage() . "<br>";
    }
}
echo "Fin<br>";
```

Prédis

Qu'affiche ce programme ?

Exécute, puis explique

Les classes `DivisionParZeroException` et `NombreNegatifException` ne contiennent aucune méthode. D'où vient alors la méthode `getMessage()` ? Indice : relis la première ligne de chacune.

Pour x = 0, la racine se calcule sans problème. Quelle méthode lance alors une exception ?

À chaque tour de boucle, combien de `catch` sont exécutés au maximum ? Comment PHP choisit-il lequel ?

Modifie

Ajoute un `catch (Exception $e)` qui affiche « Erreur quelconque : <message> », en premier, avant les deux autres. Prédis, puis exécute.

Déplace ce `catch (Exception $e)` en dernière position. Qu'est-ce que cela change ?

Supprime-le, ainsi que le `catch` de `NombreNegatifException`. Que se passe-t-il pour x = -4 ?

Quel est l'intérêt d'avoir deux types d'exception, plutôt qu'une seule `Exception` avec deux messages différents ?

✎ Ma règle : comment créer ton propre type d'exception, et comment PHP choisit-il le catch à exécuter ?

Section 5 : une exception dans un constructeur

Fichier : section5.php

```
<?php
class AgeInvalideException extends Exception {}

class Eleve {
    private $nom;
    private $age;
    function __construct($nom, $age) {
        if ($age < 0) {
            throw new AgeInvalideException("Âge invalide pour $nom :
$age");
        }
        $this->nom = $nom;
        $this->age = $age;
        echo "$nom est créé<br>";
    }
    function __toString() {
        return "$this->nom ($this->age ans)";
    }
}

$classe = array();
foreach (array(array("Zoé", 16), array("Malik", -3), array("Sam", 17)) as
$donnees) {
    try {
        $eleve = new Eleve($donnees[0], $donnees[1]);
        array_push($classe, $eleve);
    } catch (AgeInvalideException $e) {
        echo $e->getMessage() . "<br>";
    }
}
echo count($classe) . " élèves : " . implode(", ", $classe) . "<br>";
```

Prédis

Qu'affiche ce programme ? Combien d'élèves le tableau `$classe` contiendra-t-il ?

Exécute, puis explique

Pourquoi « Malik est créé » n'est-il pas affiché ?

Pourquoi Malik n'est-il pas dans le tableau ?

Modifie

Ajoute ces lignes à la fin de `section5.php` :

```
$tom = "pas encore créé";
try {
    $tom = new Eleve("Tom", -1);
} catch (AgeInvalideException $e) {
    echo $e->getMessage() . "<br>";
}
echo "Contenu de \$tom : $tom<br>";
```

Prédis : que contiendra `$tom` après le `try` / `catch` ? Exécute, puis explique.

Pourquoi est-ce une bonne chose qu'un élève avec un âge négatif ne puisse jamais exister, même un instant ?

✎ Ma règle : que se passe-t-il quand un constructeur lance une exception ?

Défi final

Crée un fichier `defi.php` : reprends-y le programme de la section 0 et réécris-le avec tout ce que tu as découvert :

- la méthode `diviser` ne fait plus aucun `echo` ;
- une division par zéro lance une exception d'un type que tu crées toi-même ;
- le programme va jusqu'au bout et affiche exactement :

```
Moyenne : 10
Impossible de calculer la moyenne : Division de 30 par zéro
Résultat : -1
```

Relis les deux défauts que tu as notés à la section 0. Sont-ils résolus ? Explique.